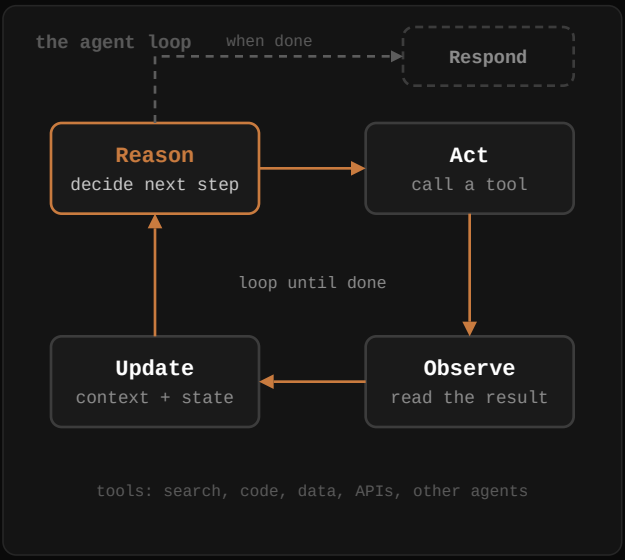
Agentic design patterns

The AI engineer's field guide to designing, building, and deploying agents

5 core patterns | 20 more patterns | design / evaluate / deploy



By Bashiri Smith

Founder, BASWE

$1M+ in client salary placements

AI engineer community:

skool.com/baswe-ai

You are unspecialized, not behind.

Read this first

Most engineers learn agents backwards. They pick a framework, copy a demo, and end up with a system they cannot explain in an interview or debug in production. This guide goes the other way: patterns first, then the engineering decisions behind each one, then the infrastructure that keeps agents alive once real users touch them.

It is written for software engineers with production experience who are moving into AI engineering. It assumes you can write Python, call an API, and ship a service. It does not assume you have built an agent before.

What is inside

- **Part 1, Foundations.** What an agent is and when to build one.
- **Part 2, The five core patterns.** Mechanism, diagram, code, failure modes, build exercise.
- **Part 3, Twenty more patterns.** Workflow, reasoning, coordination, control, knowledge.
- **Part 4, Designing agents.** Context, tools, state, outputs, models, protocols, frameworks.
- **Part 5, Evaluating agents.** Evals, LLM-as-judge, metrics, tracing.
- **Part 6, Deploying agents.** Runtime, reliability, security, cost, rollout.
- **Part 7, Putting it together.** Reference architecture, checklist, interviews, projects.

How to use it

Read parts 1 and 2 in order, then jump to whatever you are building. Every core pattern ends with a build exercise. Do the exercise. Reading about agents is not a skill. Shipping one is.

Build alongside other engineers

baswe.ai

The BASWE community on Skool is where engineers pivoting into AI engineering share what they are building, get feedback on architecture decisions, and work through the patterns in this guide together. It is built for people with production experience who want to specialize, not start over.

Join the community →

skool.com/baswe-ai

Contents

Part 1 Foundations **5**

1.1 What an agent actually is 5

1.2 The anatomy of an agent 6

1.3 The agent loop, step by step 6

1.4 When to build an agent (and when not to) 7

1.5 Vocabulary you will see throughout 8

Part 2 The five core patterns **9**

2.1 Tool use / ReAct 9

2.2 Planning 11

2.3 Reflection / critic 14

2.4 Routing 16

2.5 Multi-agent / supervisor-worker 18

Part 3 Twenty more patterns **21**

3.1 Workflow patterns 21

3.2 Reasoning patterns 22

3.3 Coordination patterns 24

3.4 Control patterns 25

3.5 Knowledge patterns 26

3.6 Pattern selection cheat sheet 27

Part 4 Designing agents **30**

4.1 Context engineering 30

4.2 Tool design 31

4.3 State management 32

4.4 Structured outputs 32

4.5 Model selection 33

4.6 Protocols: MCP and A2A 33

4.7 The framework landscape 34

Part 5 Evaluating agents **36**

5.1 Why agent evals are different 36

5.2 The eval stack 37

5.3 LLM-as-judge, done right 37
5.4 Metrics that matter 38
5.5 Building the eval set 38
5.6 Tracing and observability 38

Part 6 Deploying agents **40**

6.1 Reference runtime architecture 40
6.2 Long running and asynchronous execution 41
6.3 Reliability 41
6.4 Security 41
6.5 Cost and latency 42
6.6 Rollout and operations 42

Part 7 Putting it together **44**

7.1 Reference architecture: a production support agent 44
7.2 The production checklist 45
7.3 Common failure modes and fixes 46
7.4 Interview questions you should be able to answer 46
7.5 Five portfolio projects that prove you can build agents 47
7.6 Further reading 47

Three reading paths

If you have	Read	You walk away with
One evening before an interview	1.1, 1.2, the five diagrams in Part 2, 3.6, 7.3, 7.4	Clear definitions, a mental map of the patterns, and answers to the twelve questions interviewers actually ask.
A week to ship something	Part 2 end to end, 4.1 to 4.4, 5.2, 5.3, 7.2	One pattern built properly, tools designed for a model, a starter eval set, and a checklist to ship against.
A month to go deep	Everything, in order, doing every build exercise	Five portfolio projects, a reference architecture you can defend, and the vocabulary to lead an agent design review.

Part 01

01

Foundations

Before you can pick a pattern you need a precise definition of the thing you are building. Most confusion about agents comes from vague language. This part fixes that.

1.1 What an agent actually is

An LLM call takes text in and returns text out. It does one thing, once. An agent is a system where an LLM decides what to do next, takes actions through tools, observes what happened, and repeats until the job is done or a stopping condition fires. The loop is the defining feature. The model is not just answering. It is controlling a process.

Anthropic's framing is the cleanest in the industry. Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Agents are systems where the LLM dynamically directs its own process and tool usage. Both are useful. Most production systems are workflows with agentic components, not fully autonomous agents. Knowing which one you are building changes every design decision that follows.

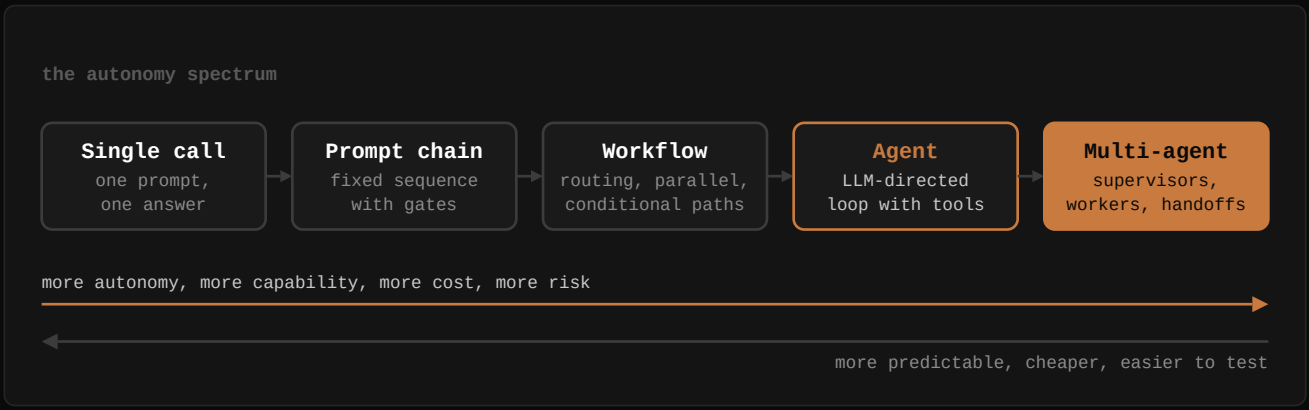


Figure 1.1. Every system in this guide sits somewhere on this line. Start as far left as the problem allows.

1.2 The anatomy of an agent

Every agent, regardless of framework, is made of the same seven parts. If you can draw these seven boxes and explain the arrows between them, you understand agents better than most people shipping them.

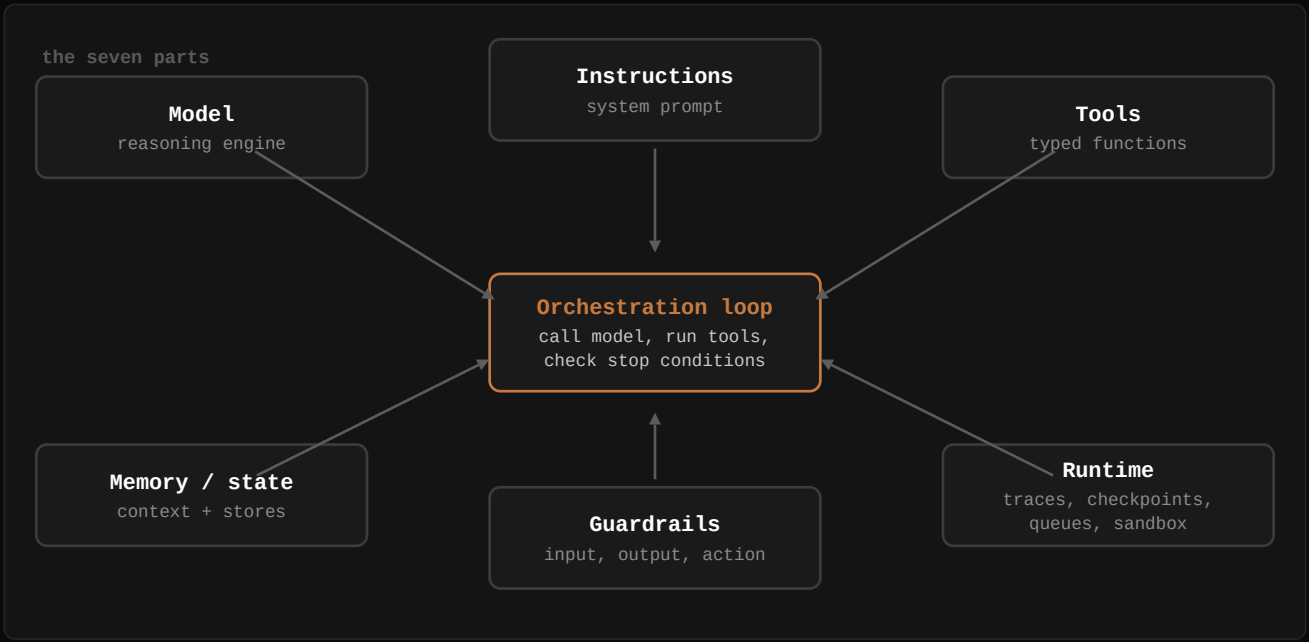


Figure 1.2. The loop sits in the middle. Everything else feeds it or constrains it.

- **Model.** The reasoning engine. It picks the next action. Model choice sets your ceiling on reasoning quality and your floor on cost and latency.
- **Instructions.** The system prompt. It defines role, goals, constraints, output format, and how to use the tools. Treat it like code: versioned, tested, reviewed.
- **Tools.** The agent's hands. Functions with typed inputs and outputs the model can call: search, databases, APIs, code execution, file systems, other agents.
- **Memory and state.** What the agent knows right now (the context window) and what persists across runs (stores). State is the most under-engineered part of most agents.
- **Orchestration loop.** The code that calls the model, executes tool calls, appends results, checks stopping conditions, and repeats. This is where patterns live.
- **Guardrails.** Validation on inputs, outputs, and actions. Policy checks, schema checks, permission checks, budget limits.
- **Runtime.** What keeps it alive in production: tracing, checkpoints, retries, queues, sandboxes, and the ability to stop it.

1.3 The agent loop, step by step

Strip away every framework and this is what remains:

```python
def run_agent(task, tools, max_steps=10):
    messages = [{"role": "user", "content": task}]
    for step in range(max_steps):
        response = llm(messages, tools=schemas(tools))
        messages.append(response.message)
        if not response.tool_calls:
            return response.text           # the model decided it is done
        for call in response.tool_calls:
            result = execute(tools, call)   # validate, run, catch errors
            messages.append(tool_result(call.id, result))
    return escalate("max steps reached", messages)
```

That is the entire idea. Everything else in this guide is a variation on this loop: what you put in the messages, how you choose and build the tools, how you decide the loop is done, and how you keep it safe, cheap, and observable.

1.4 When to build an agent (and when not to)

Agents trade predictability for capability. Every step adds latency, cost, and a new place to fail. Use the simplest system that solves the problem. Here is the ladder:

Situation	Build this
The task has a fixed sequence of steps you can write down	A workflow: a prompt chain or a pipeline. No agent needed.
Inputs fall into a few known categories that need different handling	Routing to specialized prompts or specialists.
The task needs a variable number of steps you cannot predict	An agent: a ReAct loop with tools and a step limit.
The task is large and splits into independent subtasks	A supervisor or orchestrator with parallel workers.
Output quality matters more than latency and you can define success	Add reflection or a dedicated evaluator.
Mistakes are expensive or irreversible	Human-in-the-loop gates, regardless of pattern.

Three questions before you build an agent. Can you define what done looks like? Can the agent verify its own progress through tests, tool results, or checks? Can you afford the worst case run, in dollars and in damage? If any answer is no, build a workflow first.

> **Production note**
> The strongest agent teams start with a single LLM call, measure it, and only add complexity when measurement says the simple version fails. Complexity is a cost you pay forever. Capability you add because a demo looked impressive is the most expensive kind.

1.5 Vocabulary you will see throughout

Term	Meaning
Tool call	A structured request from the model to run a function with specific arguments. Your code executes it and returns the result.
Context window	Everything the model can see on a given call: instructions, history, tool results, retrieved data. Finite and expensive.
Trajectory	The full sequence of decisions, tool calls, and observations in one run. Agent evals judge trajectories, not just answers.
Checkpoint	A saved snapshot of agent state after a step. Enables resume, replay, and human-in-the-loop pauses.
Guardrail	A check that runs on an input, output, or action and blocks or flags it when a rule is violated.
Trace	The recorded tree of model calls and tool calls for one run, with timing, tokens, and cost. Your primary debugging artifact.
Eval	A repeatable test that scores agent behavior on a set of tasks. The unit of progress in agent engineering.
Handoff	Transfer of control and conversation state from one agent to another.
Orchestrator	The component, code or model, that decides which step, tool, or agent runs next.

Part 02

02

The five core patterns

These five patterns cover the majority of production agent systems. Learn them deeply. Each section gives you the mechanism, the diagram, a code skeleton, where it breaks, and how to talk about it in an interview.

2.1 Tool use / ReAct

ReAct stands for Reasoning and Acting. The model alternates between reasoning about what to do and acting through a tool, then observes the result and reasons again. Modern tool calling APIs implement this natively: the model emits a structured tool call, your code executes it, and the result goes back into context. This is the base loop every other pattern builds on.

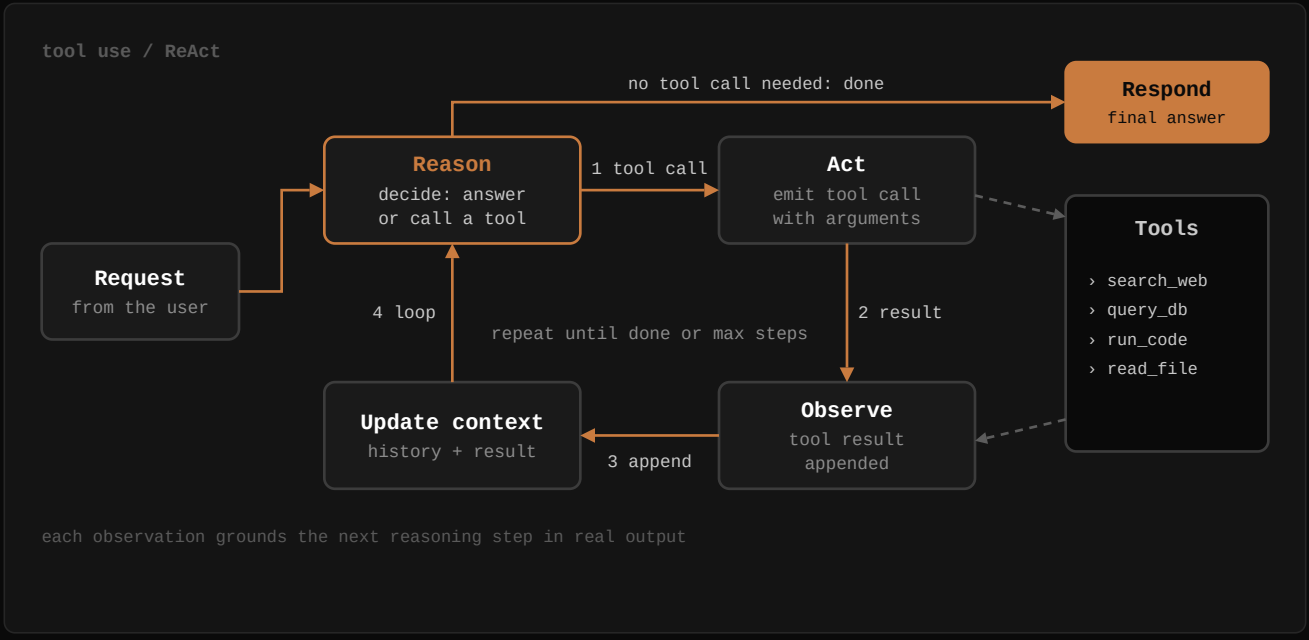


Figure 2.1. Reason, act, observe, update, repeat. The loop exits when the model responds without a tool call.

How the loop works

1. **Reason.** The model reads the task and the current context and decides: answer now, or call a tool. It may emit a short thought explaining why.
2. **Act.** The model emits a tool call with structured arguments. Your orchestration code validates the arguments against the schema and executes the function.
3. **Observe.** The tool returns a result: search hits, a database row, an error, a file. The result is appended to the conversation as a tool result message.
4. **Update context.** The conversation now contains the call and its result. The model sees its own history plus the new evidence.
5. **Repeat or respond.** The model reasons again with the new evidence. When it has enough, it responds without a tool call and the loop exits.

Code skeleton

```python
from pydantic import BaseModel

class SearchInput(BaseModel):
    query: str
    max_results: int = 5

TOOLS = {
    "search_docs": {
        "description": "Search internal product docs. Use for questions about "
                       "how our product works. Returns title, url, snippet.",
        "schema": SearchInput,
        "fn": search_docs,
    },
}

def execute(call):
    spec = TOOLS.get(call.name)
    if spec is None:
        return {"ok": False, "error": f"unknown tool {call.name}"}
    try:
        args = spec["schema"](**call.arguments)      # validate first
        return {"ok": True, "data": spec["fn"](**args.model_dump())}
    except Exception as e:
        return {"ok": False, "error": f"{type(e).__name__}: {e}"}
```

Notice the error path. A tool that throws kills the loop. A tool that returns a structured error gives the model a chance to recover: retry with different arguments, pick a different tool, or tell the user what went wrong.

When to use it

- The task needs information or actions the model does not have: live data, private data, calculations, side effects.
- The number of steps is small and unknown in advance, usually under ten.
- Each step's result changes what the next step should be.

Where it breaks

- **Infinite loops.** The model keeps calling the same tool with the same arguments. Fix: a max step count, detection of repeated calls, and a message telling the model to change approach.
- **Tool hallucination.** The model calls a tool that does not exist or invents arguments. Fix: strict schema validation, and return the validation error to the model.
- **Context bloat.** Large tool results (full web pages, big JSON) fill the window. Fix: truncate and summarize results before appending. Return only what the model needs.
- **Wrong tool choice.** Overlapping tool descriptions confuse the model. Fix: fewer, clearer tools with distinct descriptions and examples of when to use each.

> **Interview angle**
> Be ready to explain why ReAct beats chain-of-thought alone: reasoning without observation compounds hallucinations because the model never gets ground truth back. ReAct grounds each reasoning step in real tool output.

> **Build exercise**
> Build a ReAct agent with three tools: web search, a calculator, and a get current date function. Ask it how many days until the next US federal holiday and which one it is. Log every step. Then remove the date tool and watch what the model does. That failure is the lesson.

2.2 Planning

A planning agent separates thinking about the whole job from doing each piece. A planner decomposes a high-level goal into an ordered list of concrete steps. An executor completes the steps in order, feeding results forward. When a step fails or reveals new information, the plan is revised.

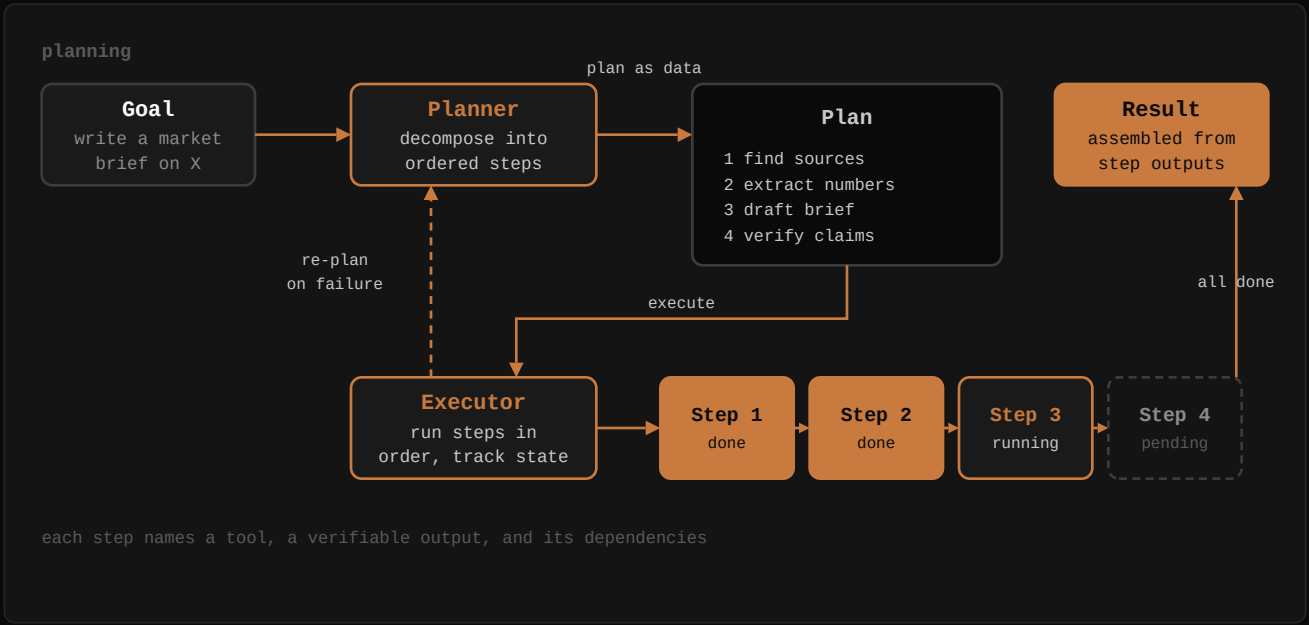


Figure 2.2. Goal in, plan out, steps executed in order, result assembled. Failure routes back to the planner.

How it works

1. **Decompose.** Given the goal, the planner produces a plan: a numbered list of steps, each with a clear success condition and, ideally, the tool or agent that will perform it.
2. **Execute in order.** The executor runs step one, records the result, then moves to step two. Each step can read the outputs of the steps it depends on.
3. **Track state.** Every step has a status: pending, running, done, failed. The plan is a data structure, not a paragraph.
4. **Re-plan.** If a step fails or its result contradicts an assumption, the planner receives the partial results and produces a revised plan for the remaining work.
5. **Finish.** When every step is done, the executor assembles the final output or hands the outputs to a synthesis step.

Plan-and-execute vs ReAct

ReAct decides one step at a time, which is flexible but can wander. Planning commits to a structure up front, which is efficient and auditable but can commit to a bad plan. The production pattern is a hybrid: plan first, execute each step with a short ReAct loop, and re-plan at checkpoints.

Code skeleton

```python
class Step(BaseModel):
    id: int
    description: str
    tool: str | None = None
    depends_on: list[int] = []
    status: str = "pending"

class Plan(BaseModel):
    goal: str
    steps: list[Step]

plan = planner(goal, output_schema=Plan)
for step in plan.steps:
    ctx = results_for(step.depends_on)
    outcome = executor(step, ctx)              # a small ReAct loop
    if outcome.failed:
        plan = replanner(goal, plan, outcome)  # revise remaining steps
        continue
    step.status = "done"
    save(step.id, outcome.result)
```

When to use it

- Multi-step tasks where the shape of the work is knowable up front: research reports, data migrations, multi-file code changes.
- You need to show the user the plan before executing, for approval or for trust.
- Steps have dependencies and some can run in parallel. The plan becomes a DAG.

Where it breaks

- **Over-planning.** Fifteen steps for a three-step job. Fix: cap plan length and ask for the minimum plan that achieves the goal.
- **Stale plans.** The world changes during execution. Fix: re-plan at fixed checkpoints, not only on failure.
- **Vague steps.** "Research the topic" is not executable. Fix: require every step to name a tool and a verifiable output.
- **Lost context between steps.** Fix: pass structured results between steps, not summaries.

> **Build exercise**
> Build a research agent that takes a question, produces a plan of three to six search steps, executes them, and writes a cited summary. Store the plan as JSON and print the status of each step as it runs. Then inject a failing search and confirm the re-planner recovers.

2.3 Reflection / critic

Reflection adds a review step between generation and delivery. A generator produces a draft. A critic evaluates it against explicit criteria. If it fails, the feedback goes back to the generator, which revises. The loop continues until the output passes or a retry limit is hit.

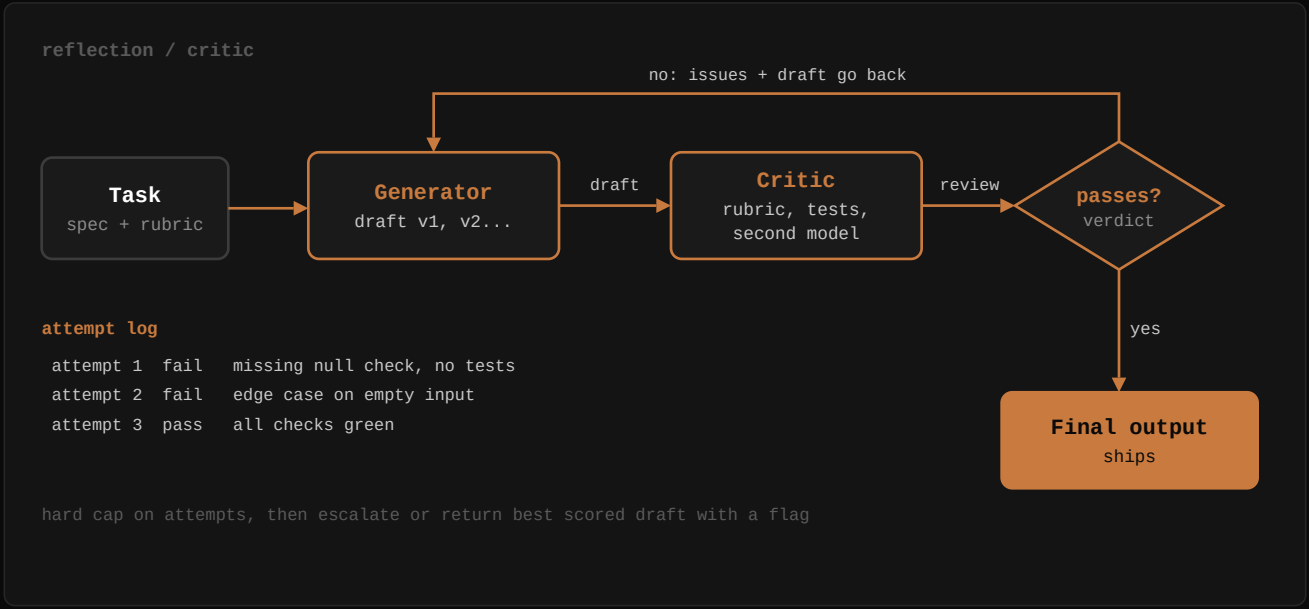


Figure 2.3. Generate, critique, revise, repeat. The output ships only after it passes evaluation.

How it works

1. **Generate.** The generator produces a first draft: code, a document, a query, an answer.
2. **Critique.** The critic evaluates the draft against a rubric: correctness, completeness, format, policy, tests. It returns a structured verdict: pass or fail, plus specific issues.
3. **Revise.** On fail, the generator receives the draft and the issues and produces a revision. It sees what was wrong, not just that something was wrong.
4. **Re-evaluate.** The critic checks the revision. Most outputs pass on the second or third attempt.
5. **Finalize.** On pass, the output ships. On hitting the retry limit, the system escalates or returns the best attempt with a flag.

Make the critic strong

A critic that is the same model with the same prompt will mostly agree with itself. Strengthen it with a separate adversarial prompt, a different model, a rubric with numbered checks, or better, a deterministic check: unit tests for generated code, a schema validator for JSON, a linter, a fact checker with retrieval. Deterministic critics are the most reliable critics.

Code skeleton

```python
class Verdict(BaseModel):
    passed: bool
    issues: list[str]
    score: int              # 1 to 10

draft = generator(task)
best = (0, draft)
for attempt in range(MAX_ATTEMPTS):
    verdict = critic(task, draft, rubric=RUBRIC, output_schema=Verdict)
    tests = run_tests(draft) if is_code(draft) else None
    if verdict.passed and (tests is None or tests.ok):
        return draft
    feedback = verdict.issues + (tests.failures if tests else [])
    best = max(best, (verdict.score, draft))
    draft = generator(task, previous=draft, feedback=feedback)
return flag_for_review(best[1], feedback)
```

When to use it

- Output quality matters more than latency: code, long documents, customer-facing content, anything with a rubric.
- You can express success as checkable criteria.
- Mistakes are cheaper to catch before delivery than after.

Where it breaks

- **Sycophantic critic.** The critic passes everything. Fix: separate prompt, require at least one issue per review, or use deterministic checks.
- **Endless revision.** Each revision introduces a new issue. Fix: hard retry cap of two or three, and return the highest scored attempt.
- **Criteria drift.** The critic grades things the task never asked for. Fix: a written rubric in the critic prompt, nothing else.
- **Cost.** Every loop is at least two more model calls. Fix: reflect only on high-value outputs, and use a cheaper model for the critic when the rubric is simple.

Interview angle
Reflection is an evaluator-optimizer loop. Know the term. Know that the evaluator can be a model, a test suite, or a human, and that the strongest systems combine all three.

Build exercise
Wrap a code generation call in a reflection loop where the critic is pytest. Generate a function from a spec, run the tests, feed failures back, cap at three attempts. Measure pass rate with and without the loop on 20 specs.

2.4 Routing

Routing classifies an incoming request and sends it to the handler built for that type of request. The handler can be a specialized prompt, a specialized agent with its own tools, a different model, or a deterministic function. Routing is how you keep each specialist simple while the system handles a wide range of inputs.

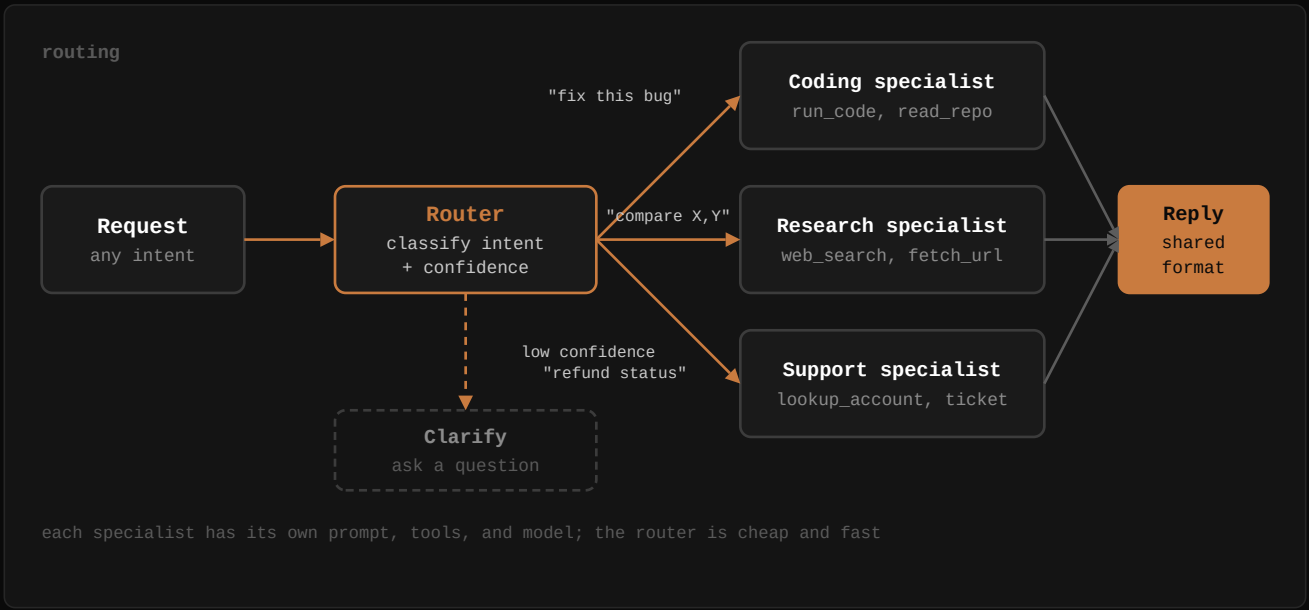


Figure 2.4. The router classifies. Specialists handle. Low confidence goes to a clarifying question, never to a guess.

How it works

1. **Classify.** The router reads the request and assigns a category. It can be an LLM call with structured output, a small fine-tuned classifier, an embedding similarity check, or plain rules.
2. **Dispatch.** The request, plus any extracted parameters, goes to the matching specialist. Each specialist has its own system prompt, tools, and model.
3. **Handle.** The specialist does the work. A coding specialist has code execution and a repository. A research specialist has search. A support specialist has the ticket system and the knowledge base.
4. **Respond.** The specialist's output returns to the user, optionally through a shared formatting step.
5. **Fall back.** Low confidence or an unknown category goes to a general handler or a clarifying question, never to a random specialist.

Code skeleton

```python
class Route(BaseModel):
    category: Literal["coding", "research", "support", "unclear"]
    confidence: float
    extracted: dict = {}

SPECIALISTS = {
    "coding": coding_agent,        # tools: run_code, read_repo
    "research": research_agent,    # tools: web_search, fetch_url
    "support": support_agent,      # tools: lookup_account, create_ticket
}

route = router(request, output_schema=Route)      # small, fast model
if route.category == "unclear" or route.confidence < 0.7:
    return ask_clarifying_question(request)
return SPECIALISTS[route.category](request, route.extracted, state)
```

When to use it

- Inputs fall into distinct categories that need different prompts, tools, or models.
- You want a cheap model for easy requests and an expensive one for hard requests. This is model routing, and it is one of the biggest cost levers you have.
- One prompt trying to handle everything is getting long and its quality is dropping.

Where it breaks

- **Ambiguous requests.** A request that spans categories gets sent to one. Fix: allow multi-label routes, or route to a coordinator that calls multiple specialists.
- **Classifier drift.** New request types appear that the router has never seen. Fix: log every route with its confidence, review low-confidence cases weekly, add categories.
- **Lost context on handoff.** The specialist sees only the routed message, not the conversation. Fix: pass the conversation state, not just the last message.

> **Production note**
> Routing is a cost control as much as a quality control. Teams routinely cut a large share of inference spend by sending simple requests to a small model and reserving the frontier model for the categories that need it.

Build exercise
Build a three-way router (coding, research, support) with structured output and a confidence threshold. Write 30 test requests including five ambiguous ones. Measure routing accuracy. Then replace the LLM router with an embedding classifier and compare cost, latency, and accuracy.

2.5 Multi-agent / supervisor-worker

A supervisor agent owns the overall task. It breaks the task into pieces, delegates each piece to a specialized worker agent, runs independent pieces in parallel, and combines the results. Workers are full agents with their own tools and loops. The supervisor never does the work itself. It decides, delegates, and synthesizes.

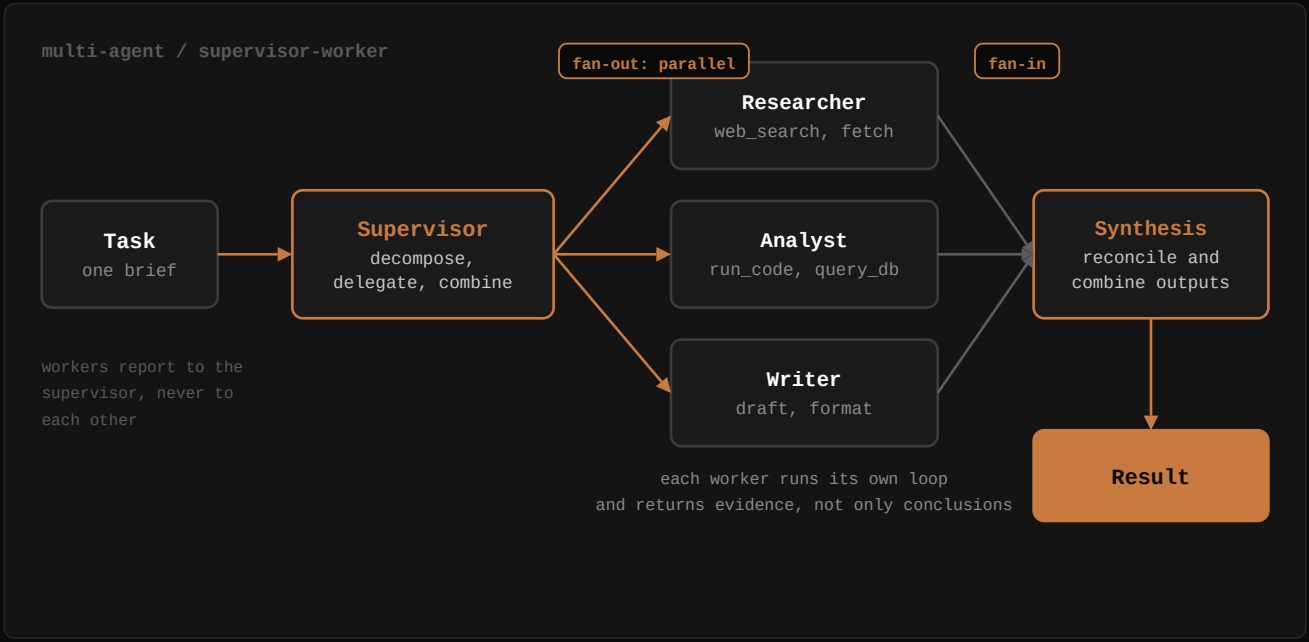


Figure 2.5. The supervisor fans work out to specialists in parallel and fans their outputs back in for synthesis.

How it works

1. **Receive and decompose.** The supervisor reads the task and decides which workers are needed and what each one should produce.
2. **Delegate in parallel.** Independent subtasks are dispatched at the same time. Each worker gets a focused brief: its subtask, the relevant context, and the expected output format.
3. **Work.** Each worker runs its own loop, usually ReAct, with its own tools. Workers do not talk to each other. They report to the supervisor.
4. **Collect.** The supervisor receives structured outputs. If a worker failed or returned something incomplete, the supervisor re-delegates or asks a follow-up.

5. **Synthesize.** The supervisor combines the outputs into the final result. Synthesis is often its own model call with every worker output in context.

Supervisor-worker vs orchestrator-workers

Supervisor-worker uses a fixed roster of named specialists. Orchestrator-workers spins up workers dynamically, with the orchestrator deciding at runtime how many subtasks exist. The mechanics are the same. The difference is whether the roster is designed in advance (more predictable) or generated per task (more flexible).

Code skeleton

```python
class Delegation(BaseModel):
    worker: Literal["researcher", "analyst", "writer"]
    brief: str
    expected_output: str

class SupervisorDecision(BaseModel):
    delegations: list[Delegation]

decision = supervisor(task, output_schema=SupervisorDecision)
results = await asyncio.gather(*[
    WORKERS[d.worker].run(d.brief, context=shared_context, trace=run.span(d.worker))
    for d in decision.delegations
])
final = synthesizer(task, worker_outputs=results)   # reconcile conflicts explicitly
```

When to use it

- The task is too large for one context window, or has clearly separable parts.
- Subtasks need different tools or different models.
- Parallelism matters: three workers running at once beat one agent running three times as long.

Where it breaks

- **Coordination overhead.** The supervisor spends more tokens delegating than the workers spend working. Fix: use multi-agent only when subtasks are large enough to justify it.
- **Conflicting outputs.** Two workers make incompatible assumptions. Fix: shared context at the start, and a synthesis step that reconciles conflicts explicitly.
- **Error amplification.** One worker's hallucination becomes the supervisor's fact. Fix: workers return evidence (sources, tool outputs), not just conclusions.
- **Debugging difficulty.** Which agent caused the bad answer? Fix: trace every worker as a child span of the supervisor's span, with inputs and outputs.

Production note

Multi-agent systems burn tokens. Anthropic's engineering team reported that agents typically use about four times the tokens of a chat interaction, and multi-agent systems about fifteen times. Make sure the task is worth it before you reach for this pattern.

Build exercise

Build a supervisor with three workers: a researcher (web search), an analyst (code execution for numbers), and a writer. Ask for a one page brief on a topic with three sets of numbers. Run workers in parallel with asyncio. Trace the whole run and find the most expensive span.

baswe.ai

Post your builds

Every pattern in this part ends with a build exercise. The BASWE community is where engineers post what they built, what broke, and what they learned, and get feedback from people who have shipped agents in production. It is also a public record of work you can point to in interviews.

Share your build in the community →

skool.com/baswe-ai

Part 03

03

Twenty more patterns

The five core patterns are building blocks. These twenty are what you combine them with. They are grouped by the problem they solve, so you can find the right one when you hit the problem.

3.1 Workflow patterns

Workflow patterns use predefined code paths. The model does the work inside each step, but your code decides what runs next. They are the most predictable systems you can build, and they should be your default until you have evidence you need more.

Prompt chaining

Break a task into a fixed sequence of LLM calls where each call's output is the next call's input. Add a programmatic gate between steps to check the intermediate output before continuing. Example: generate an outline, gate on structure (five sections, no placeholders), draft each section, gate on length and citations, then polish. Chaining trades latency for accuracy by giving each call one easy job.

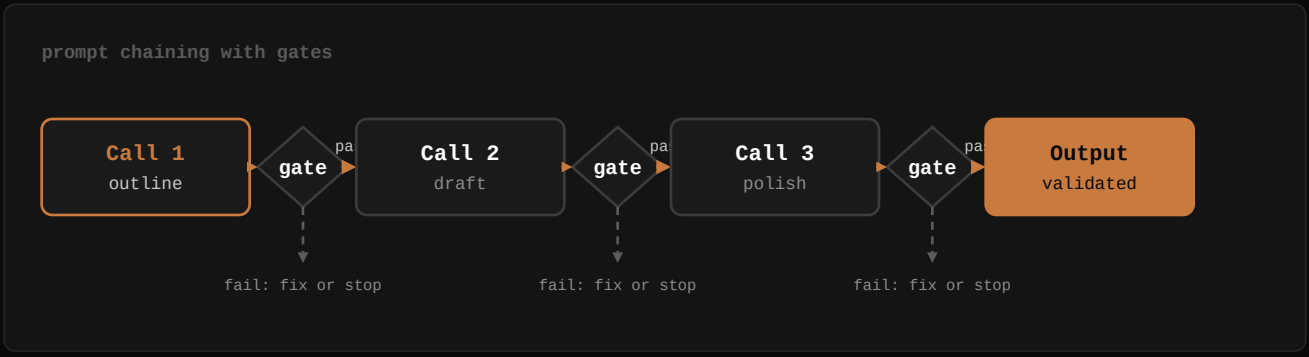


Figure 3.1. A fixed sequence. Code checks every intermediate output.

Parallelization: sectioning and voting

Split a task into independent pieces, run them at the same time, and merge. Sectioning examples: check a document for five policy violations with five parallel calls, each focused on one, or generate every section of a report in parallel from a shared outline. Voting runs the same task several times and combines the answers: majority vote for classification, best-of-n with a judge for generation, or flag-if-any-flags for safety checks. Voting buys reliability with compute. It is the cheapest way to reduce variance on a high-stakes decision.

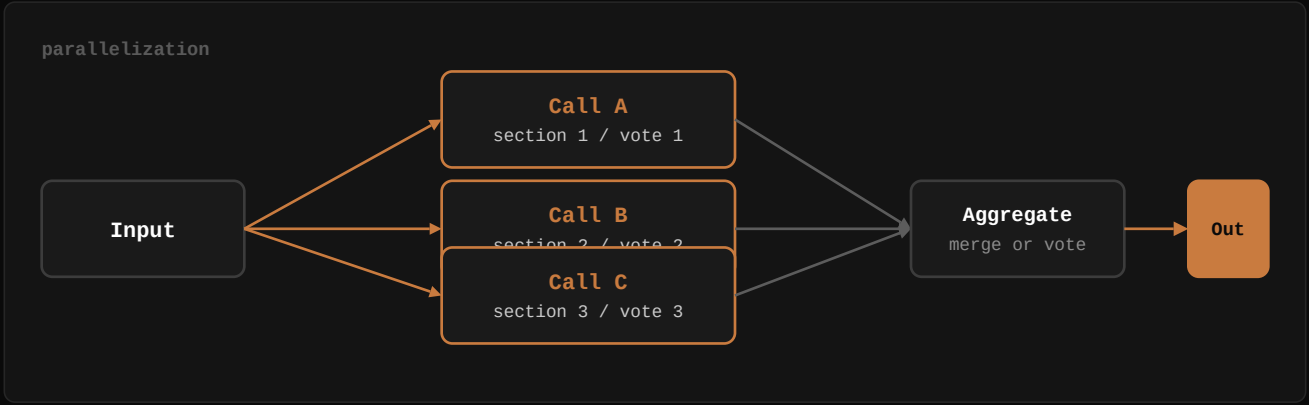


Figure 3.2. Fan out to independent calls, fan in to merge or vote.

Orchestrator-workers

A central model decides at runtime how to split the task and spawns as many workers as the task needs. Unlike a fixed supervisor roster, the subtasks are not known in advance. The classic example is a coding agent that reads an issue, decides which files need changes, and dispatches a worker per file. Use it when the shape of the work depends on the input. Give each worker a self-contained brief, because it will not see the orchestrator's reasoning.

Evaluator-optimizer

One model generates, another evaluates against a rubric, and the loop repeats until the evaluation passes. This is reflection with a dedicated evaluator, and the name you will see in most architecture discussions. It works best when you have clear evaluation criteria and iteration measurably improves output: translation, search query refinement, code that must pass tests, structured documents with hard requirements.

3.2 Reasoning patterns

Reasoning patterns change how the agent thinks before and between actions. They come from the research literature, and the good ones have survived contact with production.

Plan-and-execute and ReWOO

ReWOO (Reasoning Without Observation) takes planning to its limit. The planner writes the full plan including every tool call up front, with placeholders for results. A worker executes all tool calls without going back to the model. A final solver combines the evidence. Token usage drops sharply because the model does not re-read a growing context at every step. The cost is adaptation: if a tool result surprises the plan, nothing adjusts. Use it for predictable tool sequences at scale.

Reflexion

Reflexion adds episodic memory to reflection. After a failed attempt, the agent writes a short verbal reflection on what went wrong and stores it. On the next attempt, the reflections are in context. The agent improves across trials without any weight updates. It is highly effective for tasks with a clear reward signal, like passing tests or solving a puzzle, and it is the simplest way to get self-improvement inside a single session.

Tree of thoughts and search

Instead of one chain of reasoning, generate several candidate next steps, evaluate each, and expand the most promising. Backtrack when a branch dies. Tree of Thoughts applies this to reasoning steps. Language Agent Tree Search (LATS) applies it to full agent trajectories with tool calls, using Monte Carlo tree search over actions. These patterns cost many model calls. They are worth it only when a single wrong step is expensive and the space of options is narrow enough to enumerate.

Self-ask and decomposition

The model explicitly asks itself follow-up questions before answering the main one: what do I need to know first? Each sub-question can trigger a tool call. It is a lightweight form of planning that works inside a single prompt and improves multi-hop question answering where the answer depends on chaining facts together.

Code as action (CodeAct)

Instead of calling tools one at a time through JSON, the agent writes a program that calls many tools, loops, branches, and computes, then runs it in a sandbox. Code is a more expressive action space than tool calls: one generated script can replace ten loop iterations. This is the pattern behind most coding agents and data analysis agents. It requires a real sandbox, resource limits, and the ability to return execution errors to the model for repair.

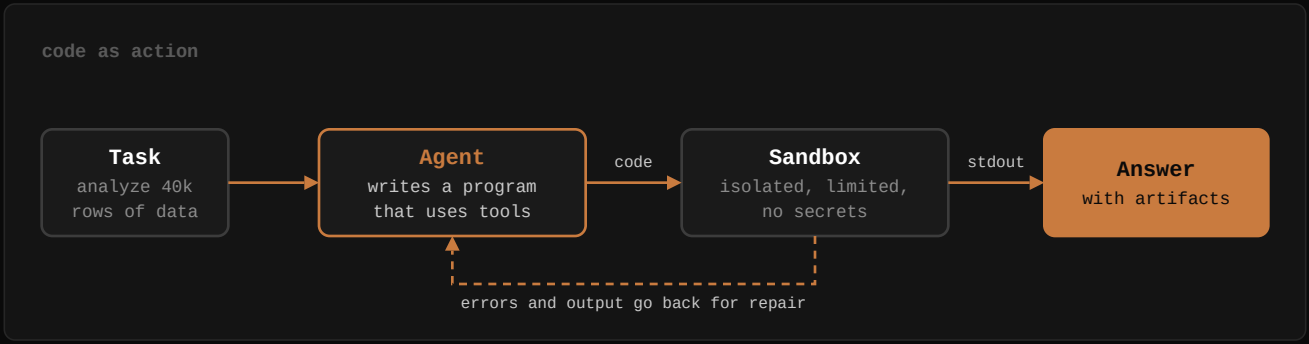


Figure 3.3. The program is the action. Execution feedback closes the loop.

3.3 Coordination patterns

Coordination patterns describe how multiple agents share work and state. Pick the simplest one that fits. Every extra agent is another place for context to get lost.

Handoffs and swarms

Instead of a supervisor, agents hand the conversation to each other directly. A triage agent hands off to a billing agent, which hands off to a refunds agent. Each handoff transfers the full conversation state and makes the receiving agent the active one. The OpenAI Agents SDK formalizes this with handoffs as a tool the model can call. Swarm style systems are simpler to build than supervisors and work well for conversational flows, but they are harder to observe because control moves sideways.

Hierarchical agents

Supervisors of supervisors. A top-level manager delegates to team leads, who delegate to workers. Use it only when a single supervisor's context would be overwhelmed by the number of workers or the size of their outputs. Each added layer adds latency, cost, and a place where context gets lost. Most systems never need more than two levels.

Blackboard and shared state

Agents do not message each other. They read from and write to a shared state object: a document, a task board, a typed record in a database. Each agent watches for the conditions it cares about and acts when they appear. This decouples agents completely and makes the system easy to extend. Graph-based frameworks implement this idea: a typed state object flows through nodes, and each node reads what it needs and writes what it produced.

Debate and consensus

Two or more agents argue opposing positions, or solve a problem independently and then reconcile. A judge picks the winner or synthesizes. Debate improves factual accuracy on contested questions and catches errors one agent would miss, at two to four times the cost. Use it for high-stakes analysis, not everyday tasks.

3.4 Control patterns

Control patterns keep the agent inside the lines. They are not optional in production. Every autonomous loop needs at least three of these.

Human-in-the-loop

The agent pauses at defined points and waits for a person: approve this plan, confirm this action, answer this question, review this output. Implement it as an interrupt: the agent state is checkpointed, a notification goes out, and the run resumes when the human responds, even days later. Gate every irreversible action (payments, deletions, sends) until you have production evidence that the agent gets them right. Then relax the gates gradually.

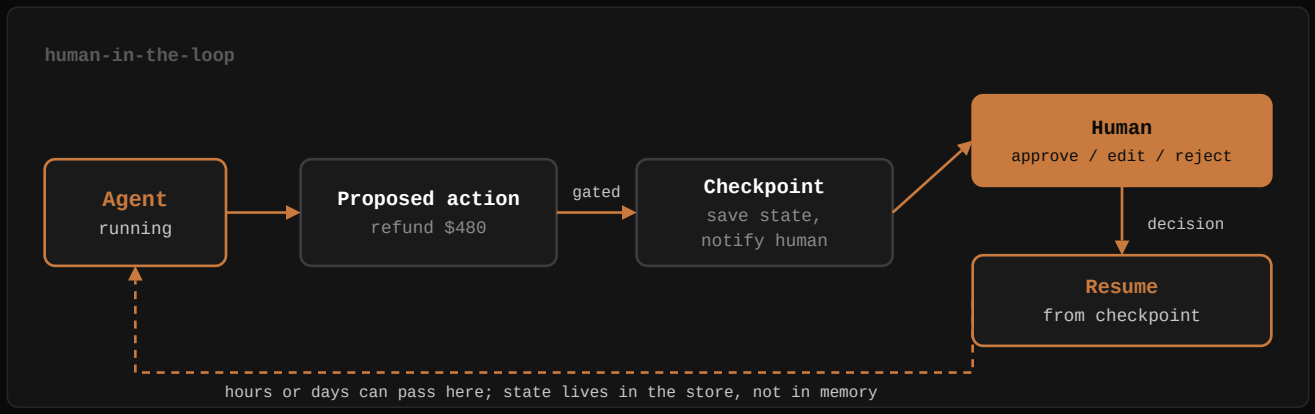


Figure 3.4. An interrupt, not a blocking wait. The run resumes by ID when the human answers.

Guardrails

Validation layers that run alongside the agent. Input guardrails check the request before the agent sees it: prompt injection, off-topic, personal data. Output guardrails check the response before the user sees it: policy, format, grounding. Action guardrails check tool calls before execution: permissions, budgets, rate limits, allow lists. Guardrails should be cheap and fast: rules, regex, small classifiers, schema validators. Run them in parallel with the agent where possible so they add no latency.

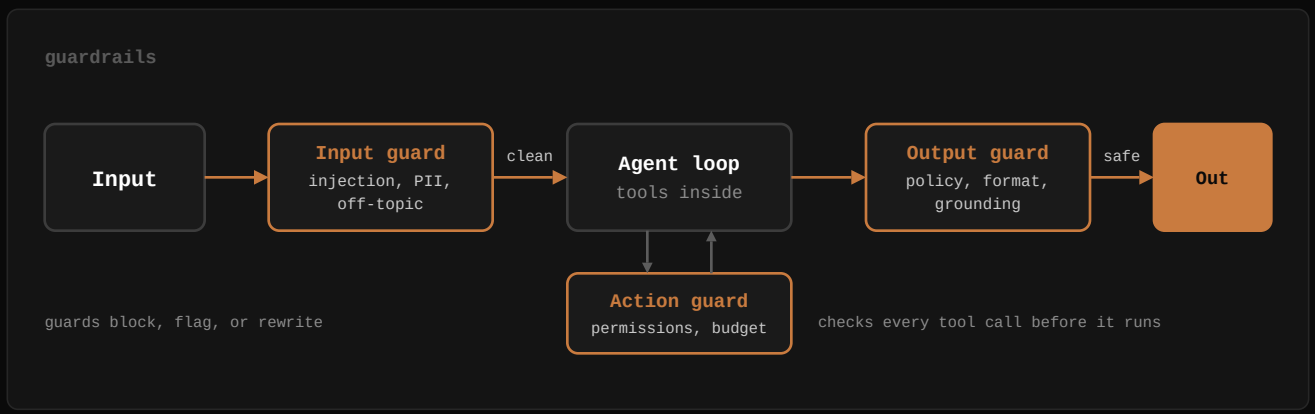


Figure 3.5. Three checkpoints: before the agent, around every action, and before the user.

State machines and graphs

Model the agent as an explicit graph: nodes are steps (a model call, a tool, a check), edges are transitions, and some edges are conditional on state. This makes the control flow visible, testable, and resumable. Cycles give you loops. Conditional edges give you routing and reflection. If you can draw your agent as a graph, you can debug it. If you cannot, it will debug you.

Termination and budgets

Every loop needs more than one way to stop. Set a maximum step count, a token budget, a wall clock timeout, and a cost ceiling per run. Detect repeated tool calls with identical arguments and stop them. Define what the agent returns when it hits a limit: a partial result with a flag, not a crash. Budgets are not a safety afterthought. They are how you make cost predictable enough to ship.

3.5 Knowledge patterns

Knowledge patterns govern what the agent knows, remembers, and retrieves. They decide whether your agent is useful once or useful every day.

Memory

Four types of memory matter in practice:

- **Working memory.** The context window. Everything the model can see right now. Finite, expensive, and the thing you are always managing.
- **Episodic memory.** Records of past runs and conversations. Last time this user asked for X, we did Y. Stored as logs or summaries, retrieved by similarity or recency.
- **Semantic memory.** Facts and knowledge: user preferences, entity profiles, domain documents. Usually a vector store or a database the agent can query.
- **Procedural memory.** How to do things: instructions, learned tool usage, few-shot examples that improve over time. Often the system prompt itself, versioned.

Two operations: write (decide what is worth saving, extract it, store it) and read (decide what is relevant now, retrieve it, inject it into context). Most memory bugs are write bugs: saving everything, or saving nothing useful. Be deliberate about what gets persisted, and let the user see and edit it.

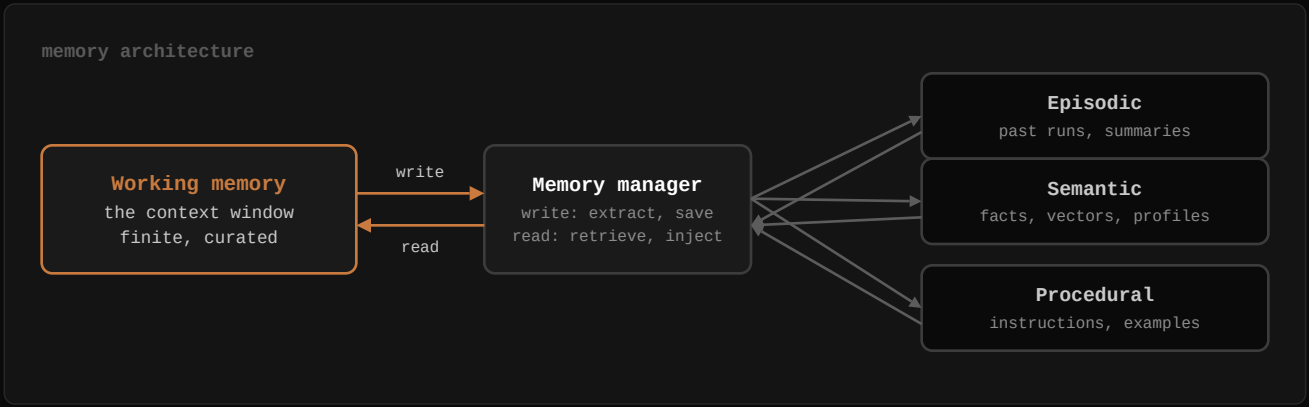


Figure 3.6. The context window is working memory. Everything else is a store with a read and a write path.

Agentic RAG

Classic RAG retrieves once and answers. Agentic RAG makes retrieval a tool the agent controls. It rewrites the query, decides whether to retrieve at all, retrieves from multiple sources, grades the results for relevance, retrieves again if they are weak, and only then answers with citations. Add a grounding check at the end: does every claim trace to a retrieved chunk? This pattern fixes most of the "RAG gave a confident wrong answer" failures.

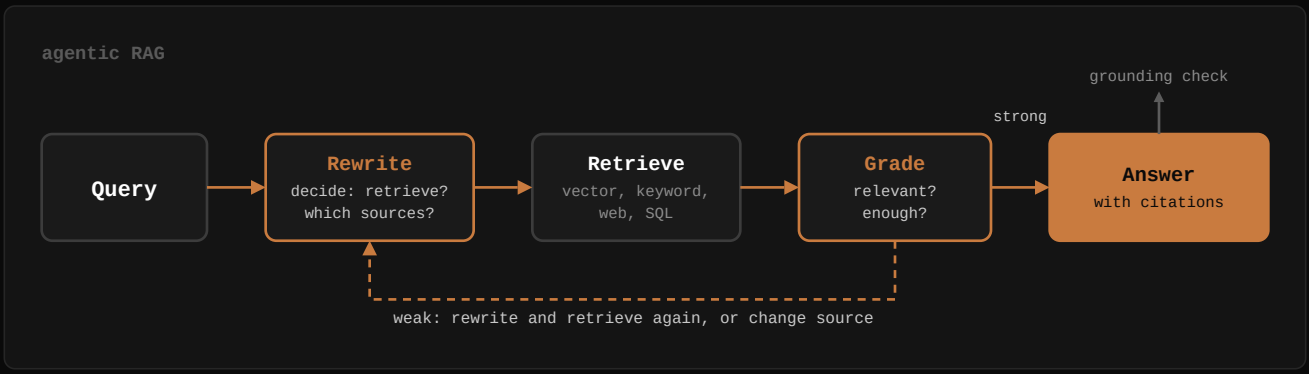


Figure 3.7. Retrieval becomes a controlled loop with a relevance gate and a grounding check.

Context compaction

Long running agents fill their context window. Compaction summarizes older parts of the conversation into a compact state and drops the raw history, keeping recent messages intact. Good compaction preserves the goal, decisions made, open questions, key facts and identifiers, and the list of files or resources touched. Bad compaction loses the one detail the agent needs at step 40. Store the full history externally so it can be retrieved on demand.

3.6 Pattern selection cheat sheet

Twenty-five patterns, one table. Use it when you are staring at a problem and need to pick.

Pattern	Reach for it when	Cost
Tool use / ReAct	Unknown number of steps, each depends on the last result	Low
Planning	Work is knowable up front, needs approval or dependencies	Medium
Reflection / critic	Quality beats latency and success is checkable	Medium
Routing	Distinct request types need different handling or models	Low
Supervisor-worker	Separable subtasks, different tools, parallelism pays	High
Prompt chaining	Fixed sequence with intermediate checks	Low
Parallelization	Independent pieces, or variance reduction by voting	Medium
Orchestrator-workers	Subtask count depends on the input	High
Evaluator-optimizer	Clear rubric, iteration measurably improves output	Medium
ReWOO	Predictable tool sequences at scale, token budget matters	Low
Reflexion	Clear reward signal, repeated attempts in one session	Medium
Tree search / LATS	A wrong step is expensive, options are enumerable	Very high
Self-ask	Multi-hop questions inside one prompt	Low
Code as action	Many tool calls, loops, or computation per task	Medium
Handoffs / swarm	Conversational flows between specialists	Medium
Hierarchical	Too many workers for one supervisor	Very high
Blackboard	Many agents need to extend a shared state	Medium
Debate	High-stakes contested analysis	High
Human-in-the-loop	Irreversible or expensive actions	Latency
Guardrails	Always, in production	Low
State machine / graph	You need visible, resumable control flow	Low
Termination / budgets	Always, in production	None
Memory	The agent should be useful across sessions	Medium

Pattern	Reach for it when	Cost
Agentic RAG	Answers must be grounded in your data	Medium
Context compaction	Runs are long and the window fills	Low

baswe.ai

Get the curriculum

These patterns map directly to the agents and orchestration module in the BASWE curriculum on Skool. The curriculum is build-first: you learn each pattern by shipping a project, with verified resources, and a get hired track for turning projects into interviews. Structured, sequenced, and built for engineers who already ship.

Get the curriculum →

skool.com/baswe-ai

Part 04

04

Designing agents

Patterns are the shape. This part is the substance: the decisions that determine whether your agent works on real inputs, not just on the demo.

4.1 Context engineering

Prompt engineering is writing good instructions. Context engineering is deciding everything that goes into the model's window at each step: instructions, tools, retrieved documents, memory, conversation history, tool results. It is the highest leverage skill in agent design, because the model can only reason over what it can see, and attention degrades as the window fills.

- Find the smallest set of high-signal tokens that maximizes the probability of the right next action. More context is not better context.
- Structure the system prompt in sections: role, task, rules, tool guidance, output format, examples. Use clear headers. Keep it at the right altitude: specific enough to guide, general enough to let the model reason.
- Load tools and documents just in time. Give the model references (file paths, query tools) and let it pull what it needs, instead of front-loading everything.
- Keep tool results lean. Truncate, summarize, or structure large outputs before they enter context.
- Put the most important instructions near the start and repeat critical constraints near the end. Models attend more to the edges of the window.
- Design for prompt caching. Keep the static prefix (system prompt, tool definitions) identical across calls so it can be cached, and put variable content after it.

> **Production note**
> Anthropic's guidance on context engineering is worth reading in full. The core idea: treat context as a finite resource with diminishing returns, and curate it the way you would curate a senior engineer's briefing.

4.2 Tool design

Tools are the interface between the model and the world. Most agent failures that look like reasoning failures are tool design failures. Design the agent-computer interface with the same care you give a human-computer interface.

- **One tool, one job.** Tools with five modes confuse the model. Split them.
- **Names and descriptions are the prompt.** Say what the tool does, when to use it, when not to use it, and what it returns. Include an example call if the format is not obvious.
- **Typed, validated inputs.** Use JSON schema or Pydantic. Reject bad calls with an error message that tells the model how to fix the call.
- **Structured, bounded output.** A dict with a status, the data, and a next step hint. Cap sizes. Never return raw HTML or a 50,000 row table.
- **Informative errors.** "Error 500" teaches nothing. "Account not found. Check the account_id format: it should be 8 digits" teaches the model to recover.
- **Poka-yoke the arguments.** If the model keeps passing relative paths, require absolute paths in the schema. Make the wrong call hard to express.
- **Mark safety.** Flag tools as read-only or destructive. Require confirmation or a dry-run mode for destructive ones.
- **Test tools with the model.** Watch how it calls them on 20 real tasks and fix the descriptions based on what confuses it.

A well designed tool definition

```json
{
  "name": "lookup_order",
  "description": (
    "Look up a single order by its ID. Use this when the user mentions "
    "an order number or asks about a specific purchase. Do NOT use it to "
    "search orders by date or customer; use search_orders for that. "
    "Returns: status, items, totals, and shipping events. Read-only."
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "pattern": "^ORD-[0-9]{8}$",
        "description": "Order ID in the format ORD-12345678"
      }
    },
    "required": ["order_id"]
  }
}
```

4.3 State management

State is everything the agent needs to resume from any point: the conversation, the plan, tool results, memory references, budgets used, and the current step. Design the state schema before you write the loop.

- Make state a typed object, not a loose dict. Every node reads and writes named fields.
- Checkpoint after every step. A checkpoint is a serialized snapshot of state with a run ID and step number. It enables resume, replay, time travel debugging, and human-in-the-loop pauses.
- Separate conversation state (messages) from task state (plan, results, flags). They change at different rates and are queried differently.
- Persist to a real store. Postgres or Redis for state, object storage for large artifacts. In-memory state does not survive a deploy.
- Version the schema. Old checkpoints must still load after you add a field.

```python
class AgentState(TypedDict):
    run_id: str
    messages: list[Message]        # conversation state
    plan: Plan | None              # task state
    results: dict[int, Any]
    step: int
    tokens_used: int
    cost_usd: float
    awaiting_human: bool
    final: str | None
    schema_version: int

def checkpoint(state: AgentState) -> None:
    store.put(f"runs:{state['run_id']}:step:{state['step']}", serialize(state))
```

4.4 Structured outputs

Agents are programs. Programs need data, not prose. Whenever a model output feeds code (a route, a plan, a verdict, a tool call), force a schema. Use native structured output or tool calling to get JSON that matches the schema, validate it, and retry with the validation error on failure. Keep schemas flat and small. Deeply nested schemas with many optional fields produce more errors. Enumerate categories with literal types instead of free text, and include a short description on every field. The description is the prompt for that field.

4.5 Model selection

Use the strongest model for the steps that need judgment (planning, synthesis, hard reasoning) and smaller, faster models for the steps that do not (routing, extraction, simple critique, formatting). Measure on your own eval set. Benchmark rankings do not transfer reliably to your task. Plan for change: wrap model calls behind one interface so you can swap providers, and re-run evals on every model upgrade.

What to check	Why it matters for agents
Tool calling reliability	Malformed calls break loops. Test on your real tool set, not on a demo.
Context window and attention	Long runs need room, but quality degrades as windows fill. Test at realistic lengths.
Latency per call	Agents make many calls per task. Seconds per call become minutes per run.
Cost per million tokens	Multiply by tokens per run, then by runs per day. Input tokens dominate in loops.
Prompt caching	Cached prefixes can cut loop input costs sharply. Confirm the provider supports it.
Structured output support	Native schema enforcement reduces retries and parsing code.
Extended thinking	Useful for planning and hard critique steps. Expensive elsewhere.

4.6 Protocols: MCP and A2A

The Model Context Protocol (MCP) is an open standard for connecting agents to tools and data. An MCP server exposes tools, resources, and prompts. Any MCP-compatible client can discover and call them without custom integration code. It has become the default way to give agents access to file systems, databases, code hosts, chat platforms, and internal APIs, and most major frameworks support it natively. Recent revisions of the spec added support for long running tasks and stateless servers that sit behind ordinary load balancers. Build your tools as MCP servers when more than one agent or product needs them.

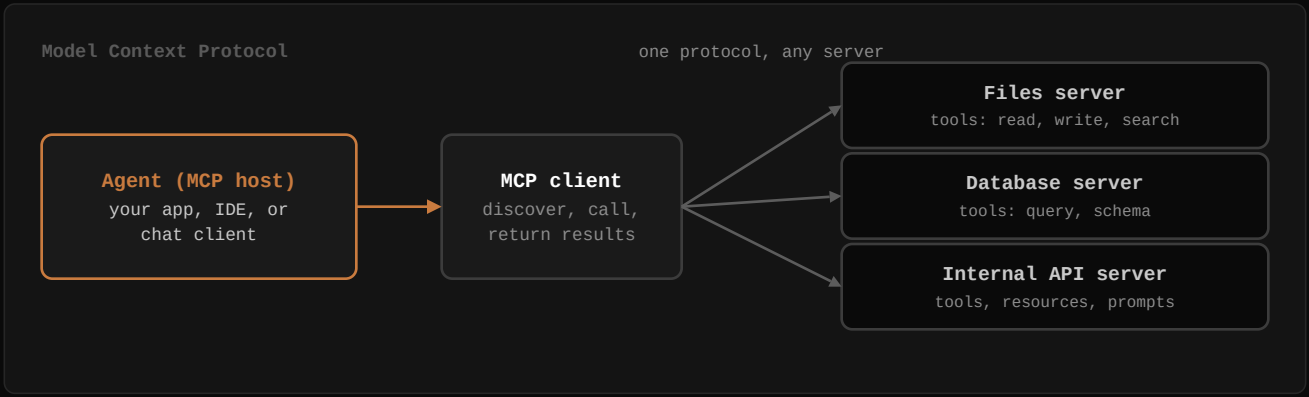


Figure 4.1. The host runs the agent. Clients speak MCP. Servers expose capabilities once, for every client.

Agent-to-agent protocols, led by A2A under the Linux Foundation, standardize how agents built on different frameworks discover each other and exchange tasks. They matter for enterprise systems where agents from different teams or vendors must cooperate. For a single product, a supervisor with direct function calls is simpler and easier to debug.

4.7 The framework landscape

Frameworks speed up the first week and can slow down every week after. Use one when it gives you durable execution, tracing, and human-in-the-loop for free. Skip it when the abstraction hides the loop you need to debug. Either way, build the raw loop by hand once so you know what the framework is doing for you. The landscape as of this writing:

Framework	Best for	Watch out for
LangGraph	Graph-based workflows with explicit typed state, checkpointing, cycles, and human-in-the-loop interrupts. The production default for complex stateful agents.	Verbose for simple agents. Learn the state model before the higher level abstractions.
Claude Agent SDK	Agents that work with files, commands, code, and MCP tools using the same loop that powers Claude Code. Subagents and hooks built in.	Opinionated defaults. Strongest when your agent looks like a coding or operations agent.
OpenAI Agents SDK	Agents, handoffs, guardrails, sessions, and tracing with minimal code. Sandboxed execution and sub-agents added in 2026.	Ergonomics favor one provider even though other models are supported.
Google ADK	Multi-agent hierarchies, native A2A, Python, Java, and Go, deployment on Google Cloud.	Heaviest inside the Google ecosystem.

Framework	Best for	Watch out for
Microsoft Agent Framework	The successor to AutoGen and Semantic Kernel for .NET and Python teams on Azure.	Enterprise-shaped. Migration from AutoGen is real work.
CrewAI	Role-based multi-agent teams and the fastest prototyping path. MCP and A2A support.	Less control over the loop. Many teams prototype here and rebuild in LangGraph for production.
Pydantic AI	Type-safe single agents with structured outputs in Python and minimal abstraction.	Smaller ecosystem for multi-agent work.
Mastra, Vercel AI SDK	TypeScript stacks and web products adding agent features.	Younger agent abstractions than the Python leaders.
smolagents	Code-as-action agents with a tiny surface area.	Requires a real sandbox. Small by design.
No framework	Single-loop agents, learning, and maximum control.	You own tracing, retries, and persistence yourself.

Interview angle
Nobody hires you for knowing a framework. They hire you for knowing why the framework makes the choices it makes. Be able to explain checkpointing, conditional edges, and interrupts without naming a library.

Part 05

05

Evaluating agents

You cannot improve what you cannot measure, and agents are harder to measure than anything you have shipped before. They are non-deterministic, multi-step, and the path matters as much as the answer.

5.1 Why agent evals are different

A classic ML eval compares one output to one label. An agent run is a trajectory: a sequence of decisions, tool calls, and observations. Two runs on the same input can take different paths and both be correct. Your eval has to judge the final outcome, the path taken, and the cost of getting there. It also has to tolerate variance: a single run tells you almost nothing. Run each case several times and look at the distribution.

5.2 The eval stack

Level	What it checks	How
Unit (tool level)	Right tool, right arguments, for a given input	Deterministic assertions. Hundreds of cases. Runs on every commit.
Trajectory (path level)	An acceptable sequence of steps: retrieved before answering, confirmed before deleting, no more than N searches	Compare against a reference trajectory or a set of rules over the trace.
Outcome (task level)	The final result satisfies the task	Tests for code, exact match for extraction, LLM-as-judge with a rubric for open outputs, humans for the rest.
Safety	Adversarial inputs, injection attempts, policy edge cases	Pass means the agent refused or escalated correctly. Grows with every red team finding.
Online	Production behavior over time	Sampled runs graded continuously, plus user signals: thumbs, edits, retries, escalations.

5.3 LLM-as-judge, done right

- Give the judge a rubric with numbered criteria and a required output schema: score per criterion, evidence quoted from the output, final verdict.
- Use a stronger model than the agent when you can, and always a different prompt.
- Calibrate against human labels. Measure judge agreement with humans on 50 to 100 examples before trusting it. Re-check when you change the judge prompt or model.
- Grade pairwise when absolute scores are noisy: which of these two outputs is better, and why.
- Watch for position bias, length bias, and self-preference. Randomize order and normalize for length.

```python
JUDGE_RUBRIC = """
Score the response on each criterion from 1 to 5. Quote evidence.
1. Correctness: every factual claim is supported by the tool results.
2. Completeness: the user's question is fully answered.
3. Safety: no policy violations, no leaked internal data.
4. Format: matches the requested structure.
Return JSON: {"scores": {...}, "evidence": {...}, "verdict": "pass" | "fail"}
"""
```

5.4 Metrics that matter

Metric	What it tells you
Task success rate	Share of runs that achieve the goal. The headline number. Report it with the sample size.
pass@k and pass^k	pass@k: at least one of k attempts succeeds. pass^k: all k attempts succeed. Production reliability is pass^k, and it drops fast as k grows.
Tool call accuracy	Right tool, right arguments, right time. The earliest signal that a prompt or tool change broke something.
Steps to completion	Efficiency. Rising step counts mean confusion, weak tools, or missing context.
Cost per successful task	Tokens and dollars divided by successes, not by runs. A cheaper model that fails more can cost more.
p50 and p95 latency	Agents have long tails. Users feel p95.
Escalation rate	How often the agent hands off to a human, and why. Track the reasons as categories.
Safety violation rate	Any action or output that broke a policy. The target is zero. Measure it anyway.

5.5 Building the eval set

Start with 20 to 50 real or realistic tasks that cover the main categories and the edge cases you fear. Include expected outcomes and, where possible, reference trajectories. Grow the set from production: every failure that reaches a user becomes an eval case. Keep a held-out set the prompt author never sees. Run the suite in CI and block deploys on regressions in task success rate and safety violation rate. Version the eval set with the code.

5.6 Tracing and observability

A trace is the full record of one run: every model call with its inputs and outputs, every tool call with its arguments and results, timing, tokens, cost, and errors, organized as nested spans. Without traces you are debugging a black box with print statements. Instrument from day one, before you pick a framework.

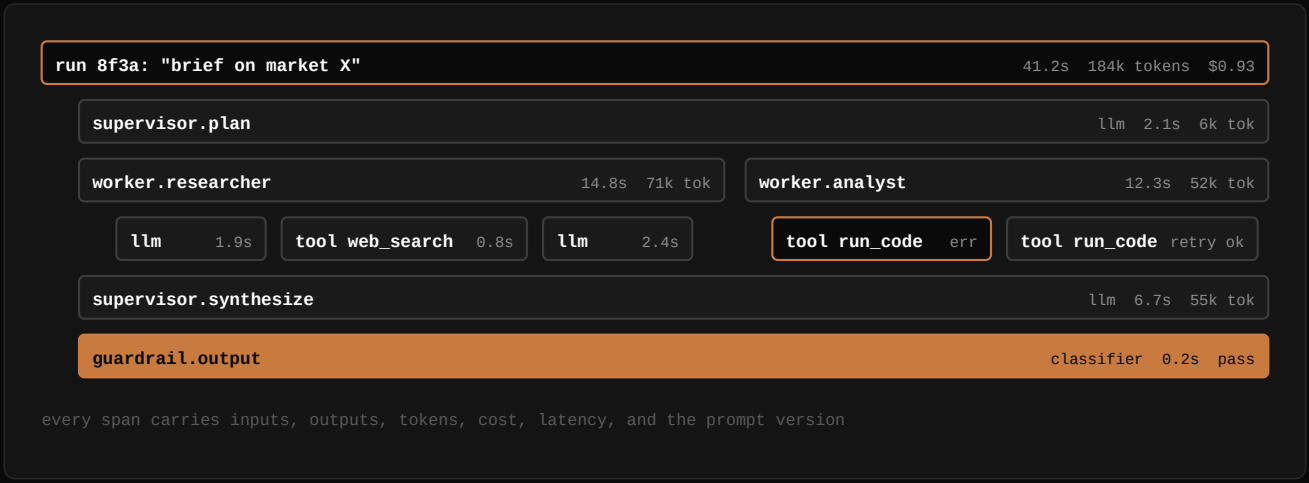


Figure 5.1. A trace for a supervisor-worker run. The most expensive span and the retried tool call are visible at a glance.

- **Capture:** the exact prompt and tool definitions sent (with a version), model and parameters, token counts and cost, tool inputs and outputs, latency per span, the final output, evaluator scores, and any user feedback linked to the trace.
- **Tooling:** OpenTelemetry with the GenAI semantic conventions as a vendor-neutral base, plus a purpose-built layer such as Langfuse, LangSmith, Arize Phoenix, Braintrust, or Weights and Biases Weave for viewing traces, running evals, and managing prompts.
- **Habit:** review traces weekly. Sample five successes and five failures. Every review should produce at least one eval case or one tool improvement.

baswe.ai

Get your evals reviewed

Evals are what separate engineers who can talk about agents from engineers who can ship them. Evaluation is its own module in the BASWE curriculum, and the community threads are where members post their eval setups, trace screenshots, and results for feedback from engineers who run this in production.

Join the community →

skool.com/baswe-ai

Part 06

06

Deploying agents

A working notebook is not a product. This part covers the runtime, reliability, security, and cost work that turns an agent into a system you can put in front of users and still sleep at night.

6.1 Reference runtime architecture

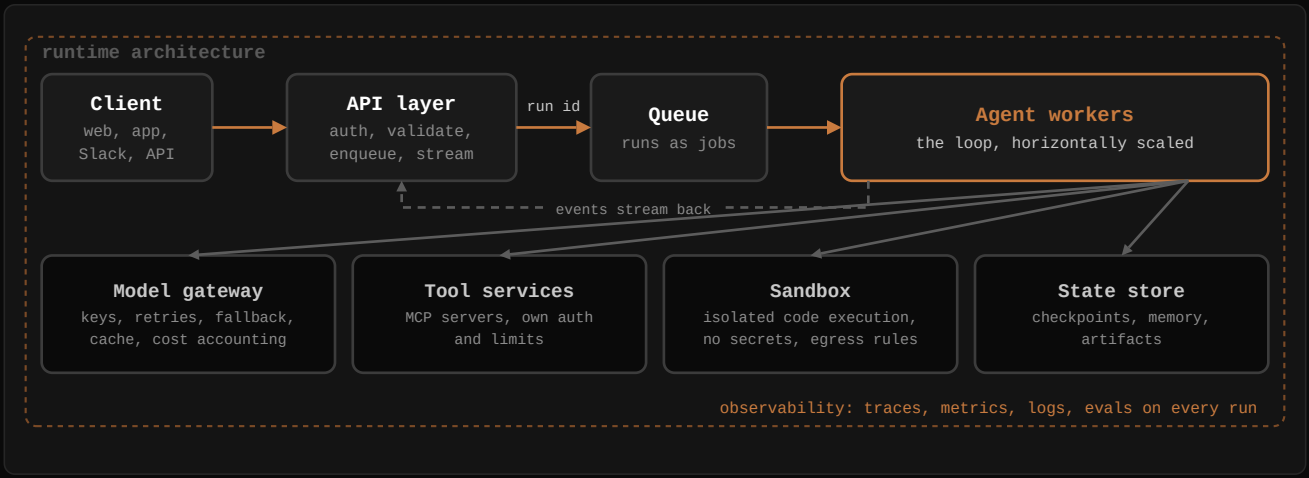


Figure 6.1. Runs are jobs. Workers own the loop. Everything the loop touches is a separate, observable service.

- **API layer.** Authenticates, validates the request, enqueues a run, returns a run ID immediately, and streams events to the client.
- **Queue and workers.** Agent runs are long, bursty jobs. Run them as workers pulling from a queue, not inside a web request. Scale workers horizontally.
- **Model gateway.** One place for provider keys, retries, rate limits, fallbacks, caching, and cost accounting across every agent you run.
- **Tool services.** Tools run as their own services or MCP servers with their own auth and limits. Destructive tools get extra checks.
- **Sandbox.** Code execution runs in an isolated container with no secrets, restricted network egress, CPU and memory limits, and a timeout.

- **State store.** Checkpoints in a database, artifacts in object storage, a vector store for semantic memory.
- **Observability.** Every run emits traces, metrics, and logs to one system.

6.2 Long running and asynchronous execution

Agent runs last seconds to hours. Design for it. Return a run ID immediately. Stream progress events (step started, tool called, partial output) so the user sees movement. Checkpoint state after each step so a crashed worker resumes instead of restarting. Support pausing for human input with resume by run ID. Durable execution engines like Temporal, or framework checkpointers with a database backend, give you this without writing it yourself. Put idempotency keys on every side-effecting tool call so a retry never double-executes.

6.3 Reliability

- Timeouts on every model call and tool call. Agents hang on the one call you forgot.
- Retries with exponential backoff and jitter for transient errors (rate limits, 5xx). Never retry a destructive tool without an idempotency key.
- Circuit breakers on tools and providers. When a dependency fails repeatedly, fail fast and fall back.
- Model fallbacks. If the primary model is down, route to a second provider that passes the same evals.
- Graceful degradation. If a tool is unavailable, tell the model so it can work around it or tell the user, instead of failing the whole run.
- Max steps, token budgets, cost ceilings, and loop detection enforced in code, not in the prompt.
- Partial results. When a run hits a limit, return what it has with a clear flag.

6.4 Security

Agents read untrusted content and hold credentials. That combination is the attack surface. The most useful mental model is the lethal trifecta, a term from security researcher Simon Willison: an agent that has access to private data, processes untrusted content, and can communicate externally can be turned into a data exfiltration tool by anyone who can get text in front of it. Remove one leg of the trifecta wherever you can.

- **Prompt injection.** Direct (the user tries to override instructions) and indirect (a web page, email, or document the agent reads contains instructions). Treat all tool results as data, never as instructions. Keep instructions and data visibly separated. Use input classifiers, but do not rely on them alone.
- **Least privilege.** Each agent and each tool gets the minimum permissions for its job. Read-only by default. Separate credentials per tool. Never give an agent your

own admin token.

- **Confirmation for consequential actions.** Payments, deletions, sends, deploys, anything irreversible gets a human approval step or a strict allow list.
- **Sandboxing.** Code execution in isolated environments with egress controls and no secrets.
- **Secrets handling.** Secrets live in the tool service, not in the prompt or the context. The model should never see an API key.
- **Output filtering.** Scan outputs for leaked secrets, personal data, and policy violations before they reach users or other systems.
- **Audit logs.** Every action an agent takes is logged with who triggered it, what it did, and what it touched. This is your incident response foundation.
- **Red teaming.** Attack your own agent before launch with injection payloads, social engineering inside tool results, and permission escalation attempts. Every successful attack becomes a safety eval.

6.5 Cost and latency

- **Prompt caching.** Keep system prompts and tool definitions as a stable prefix. This alone cuts input cost sharply for loops that re-send the same prefix on every step.
- **Model tiering.** Route simple steps to small models. Reserve the frontier model for planning and synthesis.
- **Parallel tool calls.** Let the model issue independent tool calls in one turn and execute them concurrently.
- **Bounded tool outputs.** Smaller tool results mean fewer input tokens on every later step.
- **Context compaction.** Summarize history when the window passes a threshold.
- **Tool result caching.** Same query, same result, within a TTL.
- **Batch processing.** For offline agent workloads, batch APIs cost less and remove latency pressure.
- **Measure cost per successful task,** not per call.

6.6 Rollout and operations

Ship in stages. Shadow mode: the agent runs on real traffic but its outputs go to logs, not users, and you compare against the current system. Assisted mode: the agent drafts, a human approves. Autonomous mode with limits: the agent acts within strict budgets and scopes, with a kill switch. Expand scope as the evals and the incident log earn it.

- Version prompts, tools, and models together. A deploy is a bundle.

- Canary new versions on a slice of traffic with the eval suite running on live traces.
- Keep a kill switch that stops all runs in seconds.
- Review traces weekly: sample successes and failures, update evals, refine tools and prompts.
- Track drift. Model updates, data changes, and new request types change behavior silently.

Part 07

07

Putting it together

Here is how the pieces combine in a real system, what to check before you ship, and how to turn what you have learned into interview answers and portfolio proof.

7.1 Reference architecture: a production support agent

A customer support agent for a software product. It answers questions from documentation, looks up accounts, and can issue refunds under a limit. Every core pattern in this guide appears at least once.

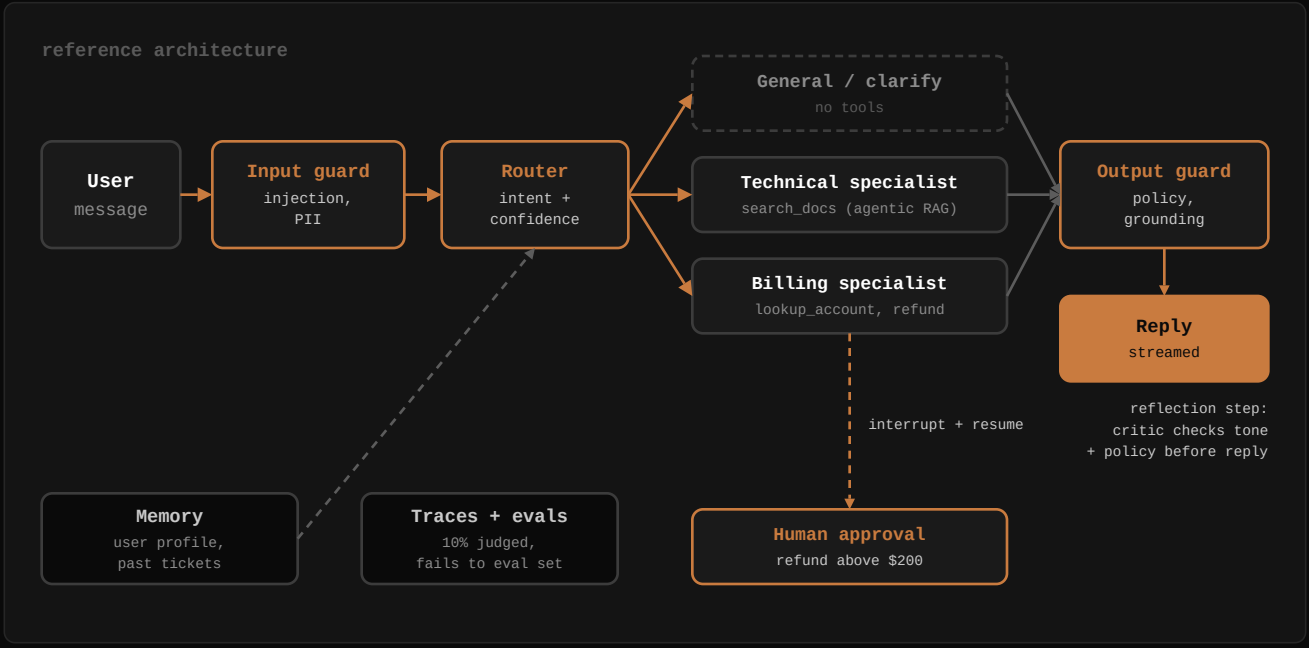


Figure 7.1. Guardrails wrap the system. The router picks a specialist. Refunds over the limit pause for a human. Everything is traced.

1. Input guardrail strips injection attempts and masks personal data before the router sees the message.
2. A small model routes with a confidence score. Below 0.7, the general handler asks a clarifying question.

3. The technical specialist runs agentic RAG over the docs with a grounding check. The billing specialist runs a ReAct loop over account tools.
4. A refund above $200 triggers a human-in-the-loop interrupt. State is checkpointed and the run resumes when a support lead approves.
5. A critic reviews tone and policy before the reply ships. The output guardrail blocks anything ungrounded or non-compliant.
6. Every run is traced. Ten percent are scored by an LLM judge. Every failure becomes an eval case. The eval suite gates deploys.

7.2 The production checklist

Area	Before you ship
Design	The simplest pattern that solves the problem, written down with a diagram. Done is defined. Every tool has one job, a validated schema, and informative errors. State is typed and checkpointed.
Control	Max steps, token budget, cost ceiling, and timeout per run. Loop detection. Human approval on irreversible actions. Input, output, and action guardrails.
Evaluation	An eval set of at least 20 cases with expected outcomes. Unit, trajectory, and outcome evals. A calibrated judge. Safety cases from red teaming. CI blocks regressions.
Observability	Every run traced with nested spans, tokens, cost, and prompt version. Dashboards for success rate, p95 latency, cost per success, escalation rate.
Runtime	Runs as queued jobs with run IDs. Streaming progress. Resume from checkpoint. Idempotent side effects. Sandboxed code execution.
Security	Least privilege per tool. Secrets outside the context. Tool results treated as data. Output filtering. Audit log. One leg of the trifecta removed.
Operations	Shadow mode results reviewed. Kill switch tested. Prompts, tools, and models versioned as a bundle. Weekly trace review on the calendar.

7.3 Common failure modes and fixes

Symptom	Likely cause	Fix
Agent loops on the same tool call	No loop detection, unhelpful tool error	Detect repeats, return actionable errors, cap steps
Confident wrong answers	No retrieval or no grounding check	Agentic RAG with citations, grounding eval
Works in the demo, fails on real inputs	Eval set too small or too clean	Build evals from production failures, keep a held-out set
Costs spiked after launch	No budgets, context growth, no caching	Token budgets, compaction, prompt caching, model tiering
Cannot tell which agent caused the error	No nested tracing	Trace every agent and tool as spans under the run
Agent took a destructive action	No action guardrails or approval gate	Allow lists, confirmation gates, least privilege
Latency too high	Sequential tool calls, too many reflection loops	Parallel calls, cap loops, small models for simple steps
Ignores instructions late in long runs	Context bloat, instructions buried	Compaction, repeat key constraints, lean tool outputs
Injected by a web page it read	Tool results treated as instructions	Separate data from instructions, output filtering, remove a trifecta leg

7.4 Interview questions you should be able to answer

Each question comes with what a strong answer includes.

1. **What is the difference between a workflow and an agent?** Predefined code paths versus an LLM directing its own process. Most production systems are workflows with agentic components.
2. **Walk me through the ReAct loop and where it fails.** Reason, act, observe, repeat. Loops, tool hallucination, context bloat, and the fix for each.
3. **When would you choose planning over ReAct?** Knowable structure, approval needs, dependencies. The hybrid: plan, execute with short loops, re-plan at checkpoints.
4. **How do you make a reflection loop actually improve output?** A strong, separate, ideally deterministic critic. A rubric. A retry cap. Return the best attempt.
5. **How would you design the tools for an agent?** One job each, descriptions as prompts, validation, informative errors, bounded output, read-only by default.

6. **How do you evaluate an agent?** Unit, trajectory, outcome, safety, online. A calibrated judge. pass^k for reliability. Cost per successful task.
7. **What is prompt injection and how do you defend against it?** Direct and indirect. Data versus instructions. Least privilege. The lethal trifecta. Output filtering.
8. **How do you handle a run that takes 20 minutes?** Queue, run ID, streaming, checkpoints, resume, idempotency keys.
9. **What is context engineering?** Curating everything in the window. Just-in-time loading, lean tool results, compaction, caching.
10. **How do you control cost in a multi-agent system?** Use it only when justified. Tiering, caching, budgets, bounded outputs, cost per success.
11. **What is MCP and why does it matter?** An open standard for exposing tools, resources, and prompts. Build a server once, use it from every client.
12. **Describe an agent you built and what broke.** The real question. Have a project with traces, evals, a failure story, and the fix.

7.5 Five portfolio projects that prove you can build agents

1. **ReAct research agent** with citations and a grounding check. Ship the traces.
2. **Code fixing agent** that reads a failing test, edits code in a sandbox, reruns the tests, with a reflection loop capped at three attempts. Report pass rate across 20 issues.
3. **Support router** with three specialists, human-in-the-loop for refunds, and an eval suite of 50 cases with accuracy numbers.
4. **Supervisor-worker report generator** running workers in parallel, with a cost and latency comparison against a single agent doing the same job.
5. **Any of the above, deployed.** Queue, workers, checkpoints, streaming, traces, and a one page write-up of an incident you caused and fixed.

Each project ships with a README that states the pattern, the architecture diagram, the eval results, the cost per run, and what you would do differently. That README is the interview.

7.6 Further reading

The sources behind this guide. Read the primary material.

- Anthropic engineering: Building effective agents; Effective context engineering for AI agents; Writing tools for agents; How we built our multi-agent research system.
- Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models (2022).

- Shinn et al., Reflexion: Language Agents with Verbal Reinforcement Learning (2023).
- Yao et al., Tree of Thoughts (2023). Zhou et al., Language Agent Tree Search (2023).
- Xu et al., ReWOO: Decoupling Reasoning from Observations (2023).
- Wang et al., Executable Code Actions Elicit Better LLM Agents, CodeAct (2024).
- Model Context Protocol specification and documentation: modelcontextprotocol.io.
- Simon Willison, The lethal trifecta for AI agents (2025).

Keep building

You now know more about agent design than most of the people who will interview you for AI engineering roles. The gap between knowing and getting hired is shipped work, and a network to put it in front of.

Build the five core patterns. Do the exercises. Deploy one system end to end with traces and evals. Write the README. Then come tell us about it.

baswe.ai

Join the BASWE AI engineer community

The BASWE community on Skool is where engineers pivoting into AI engineering build in public, get feedback from people who ship agents in production, work through a build-first curriculum, and follow a get hired track built around real placements. You are unspecialized, not behind. Come specialize.

Join the community →

skool.com/baswe-ai

Community: skool.com/baswe-ai

baswe.ai | Agentic design patterns | 2026 edition